

libbpf

- Instead of using a high level framework like BCC, one can use libbpf to build custom tools with finer control over every aspect of the program.
- libbpf is a C-based library that aims to ease eBPF programming thanks to the following features:
  - userspace APIs to handle open/load/attach/teardown of bpf programs
  - userspace APIs to interact with attached programs
  - eBPF APIs to ease eBPF program writing
- Packaged in many distributions and build systems (e.g.: Buildroot)
- Learn more at https://libbpf.readthedocs.io/en/latest/



eBPF programming with libbpf (1/2)

my_prog.bpf.c

```
#include <linux/bpf.h>
#include <bpf/bpf_helpers.h>
#include <bpf/bpf_tracing.h>

#define TASK_COMM_LEN 16
struct {
  __uint(type, BPF_MAP_TYPE_ARRAY);
  __type(key, __u32);
  __type(value, __u64);
  __uint(max_entries, 1);
} counter_map SEC(".maps");

struct sched_switch_args {
  unsigned long long pad;
  char prev_comm[TASK_COMM_LEN];
  int prev_pid;
  int prev_prio;
  long long prev_state;
  char next_comm[TASK_COMM_LEN];
  int next_pid;
  int next_prio;
};
```

- The fields to define in the `*_args` structure are obtained from the event description in `/sys/kernel/tracing/events` (see this example)



eBPF programming with libbpf (2/2)

my_prog.bpf.c

```
SEC("tracepoint/sched/sched_switch")
int sched_tracer(struct sched_switch_args *ctx)
{
  __u32 key = 0;
  __u64 *counter;
  char *file;

  char fmt[] = "Old task was %s, new task is %s\n";
  bpf_trace_printk(fmt, sizeof(fmt), ctx->prev_comm, ctx->next_comm);

  counter = bpf_map_lookup_elem(&counter_map, &key);
  if(counter) {
          *counter += 1;
          bpf_map_update_elem(&counter_map, &key, counter, 0);
  }

  return 0;
}

char LICENSE[] SEC("license") = "Dual BSD/GPL";
```



Building eBPF programs

- An eBPF program written in C can be built into a loadable object thanks to clang:

```
$ clang -target bpf -O2 -g -c my_prog.bpf.c -o my_prog.bpf.o
```

- The `-g` option allows to add debug information as well as BTF information
- GCC can be used too with recent versions
  - the toolchain can be installed with the `gcc-bpf` package in Debian/Ubuntu
  - it exposes the `bpf-unknown-none` target
- To easily manipulate this program with a userspace program based on libbpf, we need "skeleton" APIs, which can be generated with to `bpftool`



bpftool

- `bpftool` is a command line tool allowing to interact with bpf object files and the kernel to manipulate bpf programs:
  - Load programs into the kernel
  - List loaded programs
  - Dump program instructions, either as BPF code or JIT code
  - List loaded maps
  - Dump map content
  - Attach programs to hooks (so they can run)
  - etc
- You may need to mount the bpf filesystem to be able to pin a program (needed to keep a program loaded after bpftool has finished running):

```
$ mount -t bpf none /sys/fs/bpf
```



bpftool

- List loaded programs

```
$ bpftool prog
348: tracepoint  name sched_tracer  tag 3051de4551f07909  gpl loaded_at
2024-08-06T15:43:11+0200  uid 0
xlated 376B  jited 215B  memlock 4096B  map_ids 146,148
btf_id 545
```

- Load and attach a program

```
$ bpftool prog loadall trace_execve.bpf.o /sys/fs/bpf/myprog autoattach
```

- Unload a program

```
$ rm -rf /sys/fs/bpf/myprog
```



bpftool

- Dump a loaded program

```
$ bpftool prog dump xlated id 348
int sched_tracer(struct sched_switch_args * ctx):
; int sched_tracer(struct sched_switch_args *ctx)
  0: (bf) r4 = r1
  1: (b7) r1 = 0
; __u32 key = 0;
  2: (63) *(u32 *)(r10 -4) = r1
; char fmt[] = "Old task was %s, new task is %s\n";
  3: (73) *(u8 *)(r10 -8) = r1
  4: (18) r1 = 0xa7325207369206b
  6: (7b) *(u64 *)(r10 -16) = r1
  7: (18) r1 = 0x7361742077656e20
[...]
```

- Dump eBPF program logs

```
$ bpftool prog tracelog
kworker/u80:0-11  [013] d..41  1796.003605: bpf_trace_printk: Old task was kworker/u80:0, new task is swapper/13
<idle>-0          [013] d..41  1796.003609: bpf_trace_printk: Old task was swapper/13, new task is kworker/u80:0
sudo-18640        [010] d..41  1796.003613: bpf_trace_printk: Old task was sudo, new task is swapper/10
<idle>-0          [010] d..41  1796.003617: bpf_trace_printk: Old task was swapper/10, new task is sudo
[...]
```



bpftool

- List created maps

```
$ bpftool map
80: array  name counter_map  flags 0x0
    key 4B  value 8B  max_entries 1  memlock 256B
    btf_id 421
82: array  name .rodata.str1.1  flags 0x80
    key 4B  value 33B  max_entries 1  memlock 288B
    frozen
96: array  name libbpf_global  flags 0x0
    key 4B  value 32B  max_entries 1  memlock 280B
[...]
```

- Show a map content

```
$ sudo bpftool map dump id 80
[{
  "key": 0,
  "value": 4877514
  }
])
```



bpftool

- Generate libbpf APIs to manipulate a program

```
$ bpftool gen skeleton trace_sched_switch.bpf.o name trace_sched_switch
  > trace_sched_switch.skel.h
```

- We can then write our userspace program and benefit from high level APIs to manipulate our eBPF program:
  - instantiation of a global context object which will have references to all of our programs, maps, links, etc
  - loading/attaching/unloading of our programs
  - eBPF program directly embedded in the generated header as a byte array



Userspace code with libbpf

```c
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include "trace_sched_switch.skel.h"

int main(int argc, char *argv[])
{
    struct trace_sched_switch *skel;
    int key = 0;
    long counter = 0;

    skel = trace_sched_switch__open_and_load();
    if(!skel)
        exit(EXIT_FAILURE);
    if (trace_sched_switch__attach(skel)) {
        trace_sched_switch__destroy(skel);
        exit(EXIT_FAILURE);
    }

    while(true) {
        bpf_map__lookup_elem(skel->maps.counter_map, &key, sizeof(key), &counter, sizeof(counter), 0);
        fprintf(stderr, "Scheduling switch count: %d\n", counter);
        sleep(1);
    }

    return 0;
}
```



eBPF programs portability (1/2)

- Kernel internals, contrary to userspace APIs, do not expose stable APIs. This means that an eBPF program manipulating some kernel data may not work with another kernel version
- The CO-RE (Compile Once - Run Everywhere) approach aims to solve this issue and make programs portable between **kernel versions**. It relies on the following features:
  - your kernel must be built with CONFIG_DEBUG_INFO_BTF=y to have BTF data embedded. BTF is a format similar to dwarf which encodes data layout and function signatures in an efficient way.
  - your eBPF compiler must be able to emit BTF relocations (both clang and GCC are capable of this on recent versions, with the `-g` argument)
  - you need a BPF loader capable of processing BPF programs based on BTF data and adjust accordingly data access: `libbpf` is the de-facto standard bpf loader
  - you then need eBPF APIs to read/write to CO-RE relocatable variables. libbpf provides such helpers, like `bpf_core_read`
- To learn more, take a look at Andrii Nakryiko's CO-RE guide



eBPF programs portability (2/2)

- Despite CO-RE, you may still face different constraints on different kernel versions, because of major features introduction or change, since the eBPF subsystem keeps receiving frequent updates:
  - eBPF tail calls (which allow a program to call a function) have been added in version 4.2, and allow to call another program only since version 5.10
  - eBPF spin locks have been added in version 5.1 to prevent concurrent access to maps shared between CPUs.
  - Different attach types keep being added, but possibly on different kernel versions when it depends on the architecture: fentry/fexit attach points have been added in kernel 5.5 for x86 but in 6.0 for arm32.
  - Any kind of loop (even bounded) was forbidden until version 5.3
  - `CAP_BPF` capability, allowing a process to perform eBPF tasks, has been added in version 5.8



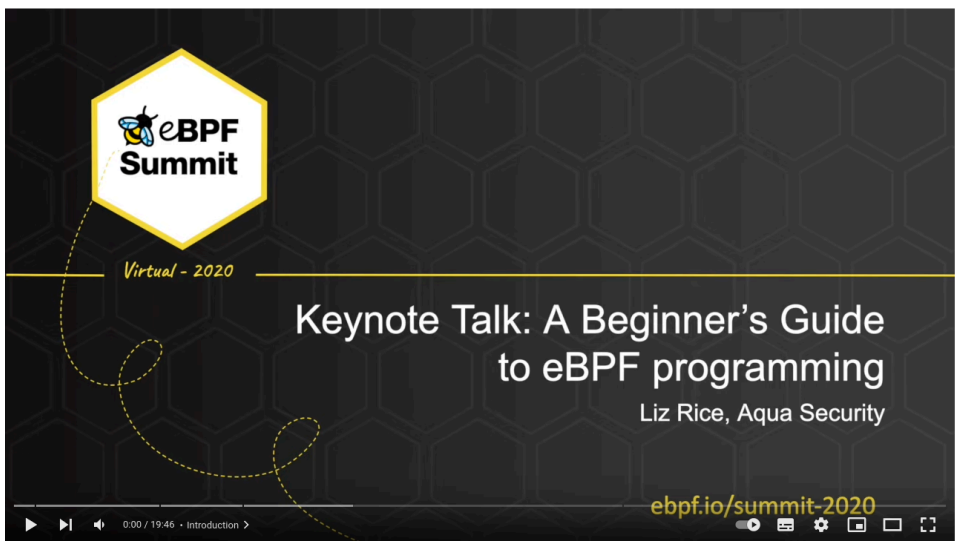
eBPF for tracing/profiling

- eBPF is a very powerful framework to spy on kernel internals: thanks to the wide variety of attach point, you can expose almost any kernel code path and data.
- In the meantime, eBPF programs remain isolated from kernel code, which makes it safe (compared to kernel development) and easy to use.
- Thanks to the in-kernel interpreter and optimizations like JIT compilation, eBPF is very well suited for tracing or profiling with low overhead, even in production environments, while being very flexible.
- This is why eBPF adoption level keeps growing for debugging, tracing and profiling in the Linux ecosystem. As a few examples, we find eBPF usage in:
  - tracing frameworks like BCC and bpftrace
  - network infrastructure components, like Cilium or Calico
  - network packet tracers, like pwru or dropwatch
  - And many more, check ebpf.io for more examples



eBPF: resources

- libbpf-bootstrap: https://github.com/libbpf/libbpf-bootstrap
- A Beginner's Guide to eBPF Programming - Liz Rice, 2020
  - Video: https://www.youtube.com/watch?v=lrSExTfS-iQ
  - Resources: https://github.com/lizrice/ebpf-beginners



Practical lab - Advanced eBPF development



Porting our custom tracing tool for embedded use case

- Converting a BCC script to libbpf
- Bringing advanced features to the tool



Choosing the right tool



Choosing the right tool

- Before starting to profile or trace, one should know which type of tool to use.
- This choice is guided by the level of profiling
- Often start by analyzing/optimizing the application level using application tracing/profiling tools (valgrind, perf, etc).
- Then analyze user space + kernel performance
- Finally, trace or profile the whole system if the performance problems happens only when running under a loaded system.
  - For “constant” load problems, snapshot tools works fine.
  - For sporadic problems, record traces and analyze them.
- If you happen to have a complex setup that you often have to bring up, it is likely a sign that you want to ease this setup with some custom tooling: scripting, custom traces, eBPF, etc